

PlanifPro

Application de gestion de planning pédagogique

Rapport final de projet

Stage 5 : Clôture du projet portfolio

Pawnee Defize

Holberton School Bordeaux

Juillet 2026

1. Contexte du projet

PlanifPro est une Progressive Web App conçue pour résoudre un problème réel et récurrent : l'organisation du planning hebdomadaire d'un professeur de musique disposant de plusieurs dizaines d'élèves, chacun ayant ses propres contraintes horaires et une durée de cours différente selon son niveau. Avant PlanifPro, cette planification se faisait manuellement, sur papier ou tableur, au prix de plusieurs jours de travail à chaque rentrée et à chaque changement d'emploi du temps.

L'application propose un flux complet : le professeur crée une classe en définissant sa période, ses jours et ses plages horaires ainsi que les contraintes de vœux ; les élèves rejoignent la classe via un code unique et soumettent leurs vœux d'horaires ; un algorithme génère trois propositions de planning que le professeur compare, ajuste et valide ; les créneaux sont ensuite exportables vers Google Calendar. Un module de suivi pédagogique permet enfin au professeur de rédiger des objectifs et des conseils après chaque cours, transmis à l'élève par notification.

Le projet a été mené en solo, sur cinq sprints, du 25 mai au 6 juillet 2026, selon une démarche Agile adaptée à un développeur unique : planification de sprint, revue et rétrospective à chaque itération, suivi sur un tableau GitHub Projects, discipline Gitflow avec pull requests systématiques et auto-revue. Le MVP a été soumis le 7 juillet, la période du 8 au 16 juillet étant consacrée aux corrections et à la préparation de la soutenance.

1.1 Déroulé des sprints

Sprint	Période	Contenu
Sprint 1	25/05 → 31/05	Fondations backend et authentification : organisation GitHub, Flask + PostgreSQL, modèles Utilisateur, Professeur, Eleve, JWT et bcrypt, premiers tests.
Sprint 2	01/06 → 07/06	Classes et vœux : modèles Classe, Creneau, Voeu, CRUD des classes avec code unique, gestion des élèves, collecte des vœux, mise en place de Flask-Migrate.
Sprint 3	08/06 → 12/06	Cœur métier et intégrations : endpoints créneaux et créneaux personnels avec portée, algorithme de génération des 3 propositions, suivi pédagogique, événements, notifications, OAuth Google et service email Brevo.
Sprint 4	15/06 → 27/06	Interface React PWA (deux rétrospectives) : authentification, tableau de bord professeur, FullCalendar, gestion des classes, propositions de planning, déplacement et échange de créneaux, suivi des vœux.
Sprint 5	29/06 → 06/07	Espace élève, finitions et déploiement (deux rétrospectives) : tableau de bord et calendrier élève, correction des bugs d'intégration, conteneurisation Docker, déploiement Render et Vercel.

Les sprints 4 et 5 ont chacun donné lieu à deux rétrospectives, l'un couvrant deux semaines de travail frontend et l'autre séparant la construction de l'espace élève du déploiement. Sept documents de rétrospective couvrent donc cinq sprints.

2. Résultats

2.1 Fonctionnalités cœur du MVP

- **Authentification et rôles** : inscription, connexion et déconnexion sécurisées par JWT et bcrypt, avec deux rôles distincts (professeur, élève / parent) ;
- **Gestion des classes** : création de classes typées (publiques, privées ou catégories personnalisées) avec période, jours, plages horaires, contraintes de vœux et code d'accès unique ;
- **Gestion des élèves** : invitation par email via Brevo, rattachement automatique par code, configuration de la durée de cours individuelle ;
- **Collecte des vœux** : lancement de la collecte, formulaire de saisie contraint, suivi des soumissions et relance des élèves en attente ;
- **Génération de planning** : production de trois propositions de planning à partir des vœux et des durées individuelles, comparaison, ajustement manuel et validation par le professeur ;
- **Gestion du calendrier** : calendrier interactif FullCalendar côté professeur et côté élève, avec modification, échange, déplacement et suppression de créneaux, et découpage par portée (période entière, jour unique, plusieurs jours) ;
- **Suivi pédagogique** : rédaction d'objectifs et de conseils par créneau, historique chronologique consultable par l'élève ;
- **Notifications** : trois canaux complémentaires : notifications in-app sur sept points de déclenchement, notifications push via Firebase Cloud Messaging, emails transactionnels via Brevo ;
- **Intégration Google Calendar** : connexion OAuth 2.0 avec PKCE, export des créneaux et import du planning personnel ;
- **Événements** : création d'événements ciblant une ou plusieurs classes ou des élèves spécifiques, avec notification automatique.

L'ensemble est déployé en production : frontend React / Vite sur Vercel, backend Flask conteneurisé sur Render, base PostgreSQL managée avec migrations gérées par Flask-Migrate, documentation d'API générée par Flask-RESTX (Swagger).

2.2 Objectifs SMART : prévu et réalisé

Objectif SMART	Période prévue	Résultat
1. Algorithme de génération de planning (Backend)	25 mai → 7 juin	Atteint
2. Interface Frontend du planning + export Google Calendar	7 juin → 14 juin	Atteint
3. Module de suivi pédagogique (Backend)	14 juin → 21 juin	Atteint
4. Interface Frontend du suivi pédagogique	21 juin → 28 juin	Atteint
5. Déploiement final (Vercel + Render)	7 juillet → 8 juillet	Atteint, MVP soumis le 7 juillet

Les cinq objectifs SMART définis dans le Project Charter ont été atteints. Le déploiement final a même été anticipé d'une journée, le MVP ayant été soumis le 7 juillet 2026. Les 25 issues de fonctionnalités ouvertes sur GitHub Projects ont toutes été fermées.

2.3 Périmètre MoSCoW : état de livraison

Le Project Charter définissait trente user stories, dont vingt-six dans le périmètre du MVP (Must have, Should have et Could have) et quatre explicitement écartées pour une version ultérieure.

Story	Description	Priorité	Statut
User-01	Professeur : inscription, connexion et déconnexion sécurisées (JWT)	Must have	Livrée
User-02	Élève / parent : inscription, connexion et déconnexion (JWT)	Must have	Livrée

Story	Description	Priorité	Statut
User-03	Professeur : écran d'onboarding à la première connexion	Must have	Non livrée
User-04	Élève / parent : écran d'onboarding à la première connexion	Must have	Non livrée
User-05	Professeur : création de classe (catégorie, période, jours, contraintes de vœux, code unique)	Must have	Livrée
User-06	Professeur : partage du code de classe et invitation par email	Must have	Livrée
User-07	Élève / parent : rejoindre une classe via son code	Must have	Livrée
User-08	Professeur : configuration de la durée de cours par élève	Must have	Livrée
User-09	Professeur : lancement de la collecte des vœux	Must have	Livrée
User-10	Élève / parent : planning personnel saisi manuellement ou importé de Google Calendar	Must have	Livrée
User-11	Élève / parent : soumission des vœux avec contraintes paramétrées par le professeur	Must have	Livrée
User-12	Professeur : suivi des vœux soumis et relance des élèves en attente	Must have	Livrée
User-13	Élève / parent : modification des vœux avant génération du planning	Must have	Livrée
User-14	Professeur : génération de 3 propositions de planning	Must have	Livrée
User-15	Professeur : option rendant la confirmation des élèves obligatoire avant validation	Must have	Reportée en V2
User-16	Élève / parent : notification et confirmation du créneau attribué	Must have	Partielle
User-17	Professeur : planning global interactif (FullCalendar)	Must have	Livrée
User-18	Professeur : export du planning vers Google Calendar	Must have	Livrée
User-19	Élève / parent : export du planning vers Google Calendar	Must have	Livrée
User-20	Professeur : modification manuelle d'un créneau avec choix de portée	Should have	Livrée
User-21	Professeur : échange des créneaux de deux élèves	Should have	Partielle
User-22	Professeur : rédaction d'objectifs et conseils par créneau, avec historique	Should have	Livrée
User-23	Élève / parent : consultation des objectifs et conseils, notifications	Should have	Livrée
User-24	Professeur : suppression d'un créneau avec choix de portée	Should have	Livrée
User-25	Professeur : création d'un événement ciblant des classes ou des élèves	Could have	Livrée
User-26	Élève / parent : consultation des événements du professeur	Could have	Livrée
User-27	Professeur : encaissement des paiements en ligne	Won't have	Hors périmètre
User-28	Professeur : messagerie instantanée et envoi de vidéos	Won't have	Hors périmètre
User-29	Élève / parent : messagerie instantanée et envoi de vidéos	Won't have	Hors périmètre
User-30	Application native (App Store / Google Play, React Native)	Won't have	Hors périmètre (PWA installable livrée)

Vingt-et-une des vingt-six user stories du périmètre ont été livrées intégralement, soit 81 %. Deux le sont partiellement et trois ne l'ont pas été. Les écarts sont assumés et documentés :

- **User-03 et User-04** : les écrans d'onboarding de première connexion n'ont pas pu être réalisés faute de temps ; leur absence n'empêche pas l'usage de l'application, les actions disponibles restant accessibles depuis les tableaux de bord.
- **User-15** : l'option rendant la confirmation des élèves obligatoire avant validation du planning a été volontairement reportée en V2. Le flux de validation directe par le professeur, plus simple, couvre le besoin du MVP.
- **User-16 (partielle)** : l'élève reçoit bien la notification de son créneau attribué et le voit s'afficher dans son calendrier ; seul le bouton de confirmation, dépendant de User-15, est reporté.
- **User-21 (partielle)** : l'échange de créneaux entre deux élèves est fonctionnel, mais le choix de portée n'est disponible que selon une méthode particulière et n'a pas la généralité prévue pour la modification et la suppression.

2.4 Indicateurs du projet

Indicateur	Valeur
User stories dans le périmètre MVP (Must / Should / Could)	26
User stories livrées intégralement	21 (81 %)
User stories livrées partiellement	2 (User-16, User-21)
User stories non livrées ou reportées	3 (User-03, User-04, User-15)
Issues de fonctionnalité fermées (GitHub Projects)	25 sur 25
Bugs d'intégration recensés	6 corrigés (#104 à #109), 1 assumé (#110)
Tests automatisés (pytest : modèles, repositories, endpoints)	217
Scripts de tests fonctionnels (curl)	9
Namespaces Flask-RESTX	13
Routes REST exposées / méthodes HTTP	56 / 84
Modèles SQLAlchemy	11 entités + EntiteBase
Migrations Alembic (Flask-Migrate)	7
Repositories / façades	11 / 5
Composants React	54 (56 fichiers .jsx avec App et main)
Canaux de notification	3 (in-app, push FCM, email Brevo)
Sprints réalisés / rétrospectives documentées	5 / 7
Environnements de production	2 (Vercel + Render)

Les 217 tests automatisés couvrent les modèles, les repositories et les endpoints de l'API, et sont complétés par neuf scripts curl reproduisant les parcours fonctionnels de bout en bout sur une base propre. La suite est exécutée avant chaque pull request.

3. Difficultés rencontrées et résolutions

Difficulté	Manifestation	Résolution
Architecture non planifiée	Les repositories et les façades n'avaient pas été inscrits au backlog des sprints, ce qui a décalé l'ensemble des sprints suivants.	Le retard a été absorbé en densifiant le sprint 3. Le pattern Façade + Repository a été conservé, son bénéfice sur la testabilité dépassant le coût du décalage.
Suppressions en cascade	Violations de contrainte de clé étrangère sur eleves, notifications et creneaux_perso lors de la suppression d'un compte ou d'une classe.	Déclaration explicite des relations cascade dans les modèles SQLAlchemy, puis rejeu du scénario complet de suppression.
Récurrence FullCalendar (#104)	La propriété endRecur étant exclusive, le dernier jour de chaque période de cours n'était pas affiché.	Ajout systématique d'un jour à date_fin dans creneauVersEventRecurrent, documenté comme règle de conversion.
Persistance du refresh token Google	Le refresh token OAuth étant conservé en mémoire du worker, l'export vers Google Calendar n'est valable qu'environ une heure après autorisation.	Limitation acceptée pour le MVP, externalisation vers Redis identifiée comme prochaine étape prioritaire.
Délivrabilité des emails (#110)	Les invitations envoyées via Brevo sont rejetées par Hotmail et Outlook, l'expéditeur reposant sur un domaine partagé.	Dette assumée : la démonstration s'appuie sur Gmail. L'achat d'un domaine et la configuration SPF / DKIM sont planifiés.
Erreurs JWT 422 intermittentes	Certains tests de garde échouaient uniquement lors d'exécutions groupées de la suite.	Réécriture des tests de garde avec des fixtures minimales. Cause racine non identifiée, tracée comme dette technique.
Développement sur deux machines	Deux bases PostgreSQL locales distinctes provoquaient des divergences de schéma et de jeux de test.	Discipline de migration : pull puis flask db upgrade à chaque changement de poste.

4. Rétrospective globale

Cette rétrospective consolide les sept rétrospectives documentées dans le wiki du dépôt. Le projet ayant été mené en solo, l'exercice porte sur la méthode de travail plutôt que sur la collaboration au sein d'une équipe.

4.1 Ce qui a bien fonctionné

- **Backend d'abord.** Concevoir l'algorithme de génération de planning et le module de suivi pédagogique côté backend avant d'aborder leur interface a évité de reconstruire l'API après coup. À la fin du sprint 3, toutes les fonctionnalités du MVP étaient couvertes côté API.
- **La conception amont a servi.** Les diagrammes de séquence produits au Stage 3 ont permis d'implémenter les routes d'authentification sans hésitation sur les cas d'erreur à couvrir.
- **Discipline Gitflow.** Les branches feature/ et fix/ fusionnées par pull request dans develop, puis dans main, avec auto-revue systématique et commits conventionnels, ont maintenu un historique lisible et permis d'isoler chaque régression.
- **Tests continus.** Écrire les tests au fil des fonctionnalités plutôt qu'à la fin a permis de détecter tôt des bugs sérieux, notamment l'absence de hachage du mot de passe lors d'une modification de profil.
- **Suivi Agile.** Le tableau GitHub Projects, les revues de sprint et les rétrospectives ont fourni une visibilité constante sur le reste à faire et ont facilité les arbitrages de périmètre.
- **Architecture en couches.** Le pattern Façade + Repository, adopté dès le sprint 1, a isolé la logique métier et rendu les tests d'API lisibles.

4.2 Difficultés et manière dont elles ont été traitées

- **Une couche d'architecture oubliée au backlog.** Les repositories et les façades n'avaient pas été inscrits au backlog. Cet oubli, identifié en rétrospective du sprint 2, a décalé tous les sprints suivants et a dû être absorbé en densifiant le sprint 3.

- **Charge de fin de projet.** La dernière ligne droite, consacrée aux fonctionnalités restantes, aux tests de bout en bout et à l'intégration Google Calendar, a été la plus dense du projet. Le déploiement a néanmoins été livré avec un jour d'avance.
- **Environnements multiples.** Le développement sur deux machines a introduit des divergences de schéma que la discipline de migration a corrigées, mais au prix d'un temps de synchronisation récurrent.
- **Intégrations tierces.** L'intégration OAuth 2.0 avec PKCE a demandé plus de temps que prévu et laisse une limitation connue sur la persistance du refresh token. La délivrabilité des emails Brevo vers Hotmail et Outlook reste une dette assumée.

4.3 Axes d'amélioration pour les prochains projets

- **Planifier l'architecture, pas seulement les fonctionnalités.** Le backlog listait les fonctionnalités, pas les couches techniques qui les portent. Faire figurer explicitement repositories, façades et services dans le découpage aurait évité le décalage du sprint 2.
- **Répartir l'effort de test.** Réserver une part fixe de chaque sprint aux tests et aux corrections plutôt que de les concentrer en fin de projet.
- **Anticiper les intégrations externes.** Traiter les intégrations tierces, plus coûteuses en temps que le code métier, dès les premiers sprints.
- **Sécuriser dès le déploiement.** Externaliser la gestion des secrets et restreindre le CORS dès la première mise en production, au lieu de les traiter comme de la dette.
- **Formaliser les décisions de périmètre.** Ne repousser une user story qu'après une décision explicite et tracée, comme cela a été fait pour User-15, plutôt que par manque de temps, comme pour User-03 et User-04.

5. Leçons apprises

- **La documentation technique amont paie.** Un travail de conception soigné : user stories priorisées en MoSCoW, diagrammes de classes, diagrammes de séquence, permet de développer sans revenir en arrière. L'essentiel de la documentation technique s'est retrouvé tel quel dans le code.
- **Un backlog doit décrire la structure, pas seulement le résultat.** Un backlog qui ne décrit que les fonctionnalités visibles sous-estime systématiquement la charge. Les couches invisibles à l'utilisateur, repositories et façades, représentaient un sprint entier de travail non planifié.
- **Une décision de périmètre vaut mieux qu'un abandon subi.** Une fonctionnalité reportée par décision assumée, comme User-15, laisse une trace claire dans le rapport et dans le backlog V2. Une fonctionnalité abandonnée par manque de temps laisse une dette invisible.
- **Les tests détectent ce que la relecture manque.** Le hachage manquant dans `mettre_a_jour_utilisateur` n'a été révélé que par un test. Sans suite automatisée, la faille serait partie en production.
- **Les bibliothèques imposent leurs contraintes.** Les problèmes les plus coûteux du projet, récurrence FullCalendar exclusive, classes Tailwind non générées au build, refresh token OAuth non persistant, délivrabilité Brevo, venaient tous du comportement d'un outil tiers et non du code métier. Il faut budgéter ce temps d'apprentissage.
- **La cohérence linguistique compte.** Nommer le code, les commentaires, les docstrings et les messages d'erreur en français avec accents, sur toute la base de code, a réduit la charge mentale et rendu les revues plus rapides.
- **L'auto-revue reste utile.** Sur un projet solo, la revue de pull request n'apporte pas de regard extérieur, mais impose un temps de relecture froide qui a intercepté plusieurs erreurs.
- **Assumer la dette technique.** La dette technique doit être listée, pas seulement connue. Le fait de la tenir à jour a permis de la présenter comme un plan d'action plutôt que comme un oubli.

6. Prochaines étapes

6.1 Dette technique identifiée

- Externaliser le refresh token Google OAuth vers Redis pour rendre l'export valable au-delà d'une heure et compatible avec un environnement multi-workers.
- Acquérir un domaine et configurer SPF et DKIM afin de rétablir la délivrabilité des emails d'invitation vers Hotmail et Outlook (#110).

- Restreindre la configuration CORS de l'ensemble des origines à l'URL de production Vercel.
- Effectuer une rotation complète des secrets Google, Brevo et Firebase.
- Identifier la cause racine des erreurs JWT 422 intermittentes lors des exécutions groupées de la suite de tests.
- Rendre le tableau de bord professeur pleinement responsive, sur le modèle du pattern drawer déjà en place côté élève.

6.2 Évolutions fonctionnelles (V2)

- Option de confirmation obligatoire des créneaux par les élèves avant validation du planning final (User-15 et User-16).
- Généralisation du choix de portée à l'échange de créneaux (User-21).
- Écrans d'onboarding à la première connexion (User-03 et User-04).
- Encaissement des paiements en ligne (User-27).
- Messagerie instantanée entre professeurs et élèves (User-28 et User-29).
- Application mobile native via React Native (User-30).

7. Annexes — liens du projet

- **Dépôt source** : <https://github.com/Pawnee33/PlanifPro>
- **Application en production** : <https://planif-pro.vercel.app>
- **API et documentation Swagger** : <https://planifpro-back.onrender.com/api/v1/>
- **Sprint planning (GitHub Projects)** : <https://github.com/users/Pawnee33/projects/2/views/2>
- **Suivi des bugs (GitHub Issues)** : <https://github.com/users/Pawnee33/projects/2/views/1>
- **Sprint Reviews** : https://docs.google.com/document/d/1_3C8nWxGPYosWEBdRuBM4uCvyTfjU1cXHybenF8ziWQ/edit?usp=sharing
- **Profil LinkedIn** : <https://www.linkedin.com/in/pawnee-defize/>

Les rétrospectives des cinq sprints sont consignées dans le wiki du dépôt. L'ensemble des livrables du Stage 4 est référencé dans le fichier STAGE4_LIVRABLES.md à la racine du dépôt.